</codewise>

MERN Stack Practical Task

Full-Stack Development Assessment

Overview

Welcome! This task is designed to assess your practical skills across the MERN stack (MongoDB, Express, React, Node.js). Please read through the entire brief before starting.

Time budget: 2.5–3 hours

Submission: Push your code to a public or private GitHub repository (add us as a collaborator if private) and share the link.

The Task: Personal Task Manager

Build a simple task management application where a user can create, organize, and track personal tasks.

Data Model

Design and implement a Task with at minimum:

- `title` (string, required)
- `description` (string, optional)
- `priority` (enum: low | medium | high)
- `status` (enum: todo | in_progress | done)
- `dueDate` (date, optional)
- `createdAt` (timestamp)

Feel free to extend this if it makes sense for your design — just document any additions in your README.

Part 1 — Backend (Node.js + Express + MongoDB)

Build a REST API with the following endpoints:

Method	Endpoint	Description
`POST`	`/tasks`	Create a new task
`GET`	`/tasks`	List tasks — support filtering by status and sorting by dueDate or priority
`GET`	`/tasks/:id`	Get a single task
`PUT`	`/tasks/:id`	Update a task (any field)
`DELETE`	`/tasks/:id`	Delete a task

Requirements:

- Validate incoming data — reject bad requests with a 400 and a clear error message, not a server crash.
- Use a proper error-handling middleware (no raw stack traces returned to the client).
- Structure your code with a clear separation of concerns (e.g., routes, controllers, models) — avoid putting all logic in one file.

Part 2 — Frontend (React)

Build a single-page interface that:

1. Displays all tasks, grouped or filterable by status (todo, in_progress, done).
2. Lets the user create a new task via a form.
3. Lets the user update a task's status (e.g., a dropdown or drag-style status change) and see the UI reflect it immediately.
4. Lets the user delete a task.
5. Shows a reasonable loading state and handles API errors gracefully (e.g., a failed request shouldn't leave the UI stuck or silently fail).

Note: We're not evaluating visual design polish — a clean, functional UI is enough. We care more about correct data flow and state management.

Part 3 — Short Write-Up

In a NOTES.md file, briefly answer (2–3 sentences each):

1. What would you change about your data model or API design if this needed to support multiple users, each with their own private task list?
2. If the task list could grow to thousands of items, what's the first performance concern you'd address, and how?
3. Did you use any AI coding tools during this task? If so, describe one instance where you changed or rejected a suggestion it gave you, and why.

What We're Looking For

- **Correctness:** Does the app do what it says? Do the edge cases (empty title, invalid status, non-existent ID) behave sensibly?
- **Code structure:** Is the code organized in a way that would be easy for a teammate to extend?
- **Frontend/backend integration:** Does the UI stay in sync with the server state after actions, without needing a manual refresh?
- **Communication:** Is your README clear enough that someone else could set up and run your project without asking you questions?

Submission Checklist

- Backend and frontend both run locally with clear setup instructions in README.md
- .env.example provided if you use environment variables (do not commit real secrets)
- NOTES.md with your write-up answers
- Git history reflects incremental progress (please don't submit a single squashed commit)

Good luck — we're looking forward to seeing what you build!